```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
print("Loaded Library")
```

```
Loaded Library
```

```
#load the Data from Dataset Drive
df = pd.read_csv("/content/Capstone1_part2_tesla/Tesla - Deaths.csv")
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 307 entries, 0 to 306
Data columns (total 24 columns):
 #   Column                                                      Non-Null Count  Dtype  
---  ------                                                      --------------  -----  
 0   Case #                                                      294 non-null    float64
 1   Year                                                        294 non-null    float64
 2   Date                                                        294 non-null    object 
 3    Country                                                    294 non-null    object 
 4    State                                                      294 non-null    object 
 5    Description                                                295 non-null    object 
 6    Deaths                                                     299 non-null    float64
 7    Tesla driver                                               294 non-null    object 
 8    Tesla occupant                                             290 non-null    object 
 9    Other vehicle                                              295 non-null    object 
 10   Cyclists/ Peds                                             296 non-null    object 
 11   TSLA+cycl / peds                                           297 non-null    object 
 12   Model                                                      296 non-null    object 
 13   Autopilot claimed                                          281 non-null    object 
 14   Verified Tesla Autopilot Deaths                            297 non-null    object 
 15   Verified Tesla Autopilot Deaths + All Deaths Reported to NHTSA SGO  296 non-null  object 
 16  Unnamed: 16                                                 292 non-null    object 
 17  Unnamed: 17                                                 289 non-null    object 
 18   Source                                                     297 non-null    object 
 19   Note                                                       9 non-null      object 
 20   Deceased 1                                                 87 non-null     object 
 21   Deceased 2                                                 17 non-null     object 
 22   Deceased 3                                                 4 non-null      object 
 23   Deceased 4                                                 0 non-null      float64
dtypes: float64(4), object(20)
memory usage: 57.7+ KB
```

```
df.isnull().sum()
```

	0
Case #	13
Year	13
Date	13
Country	13
State	13
Description	12
Deaths	8
Tesla driver	13
Tesla occupant	17
Other vehicle	12
Cyclists/ Peds	11
TSLA+cycl / peds	10
Model	11
Autopilot claimed	26
Verified Tesla Autopilot Deaths	10
Verified Tesla Autopilot Deaths + All Deaths Reported to NHTSA SGO	11
Unnamed: 16	15
Unnamed: 17	18
Source	10
Note	298
Deceased 1	220
Deceased 2	290
Deceased 3	303
Deceased 4	307

dtype: int64

Unnamed : 16 & Unnamed : 17 Contain weblink which is not useful Source: contains web link Note: contains additonal info Deceased 1 2 3 and 4 contain name of the deceased which is irrelevent to the analysis Case # not required Year can be derived from date

```
df.shape
```

```
(307, 24)
```

```
drop_columns=['Case #','Year','Unnamed: 16','Unnamed: 17',' Source ' , ' Note ',' Deceased 1 ',' Deceased 2 ',' Deceased 3 ','

df.drop(columns=drop_columns, inplace=True)
```

```
df.shape
```

```
(307, 15)
```

```
df.drop(columns = ' Verified Tesla Autopilot Deaths + All Deaths Reported to NHTSA SGO ', inplace = True)
```

```
df.columns
```

```
Index(['Date', ' Country ', ' State ', ' Description ', ' Deaths ',
       ' Tesla driver ', ' Tesla occupant ', ' Other vehicle ',
       ' Cyclists/ Peds ', ' TSLA+cycl / peds ', ' Model ',
       ' Autopilot claimed ', ' Verified Tesla Autopilot Deaths '],
      dtype='object')
```

```
cols = df.columns[5:]
for col in cols:
  if col != ' Model ':
```

```
    print(col)
    df[col] = df[col].fillna("-")
    df[col] = df[col].str.strip()
    df[col] = df[col].replace("-", "0")
    df[col] = df[col].astype(int)
```

```
Tesla driver
Tesla occupant
Other vehicle
Cyclists/ Peds
TSLA+cycl / peds
Autopilot claimed
Verified Tesla Autopilot Deaths
```

```
df.head()
```

	Date	Country	State	Description	Deaths	Tesla driver	Tesla occupant	Other vehicle	Cyclists/ Peds	TSLA+cycl / peds	Model	Autopilot claimed	Verified Tesla Autopilot Deaths
0	1/17/2023	USA	CA	Tesla crashes into back of semi	1.0	1	0	0	0	1	-	0	0
1	1/7/2023	Canada	-	Tesla crashes	1.0	1	0	0	0	1	-	0	0

```
df.shape
```

```
(307, 13)
```

```
df.dropna(inplace=True)
```

```
df.shape
```

```
(294, 13)
```

Change the Variable names according to python readable

```
#remove the spaces
df.columns=df.columns.str.strip()
```

```
#replace the empty,replace + to _, replace / to _
df.columns=df.columns.str.replace(" ","").str.replace("+","_").str.replace("/","_")
#
```

```
df.columns
```

```
Index(['Date', 'Country', 'State', 'Description', 'Deaths', 'Tesladriver',
       'Teslaoccupant', 'Othervehicle', 'Cyclists_Peds', 'TSLA_cycl_peds',
       'Model', 'Autopilotclaimed', 'VerifiedTeslaAutopilotDeaths'],
      dtype='object')
```

```
#rename the Columns to smaller ones
df.rename(columns = {"Autopilotclaimed":"Claimed", "VerifiedTeslaAutopilotDeaths":"VTAD",
                     "Teslaoccupant" : "Tesla_Occupant", "Othervehicle":'Other_Vehicle',
                     "Tesladriver": "Tesla_Driver"}, inplace = True)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 294 entries, 0 to 293
Data columns (total 13 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Date            294 non-null    object
 1   Country         294 non-null    object
 2   State           294 non-null    object
 3   Description     294 non-null    object
 4   Deaths          294 non-null    float64
 5   Tesla_Driver    294 non-null    int64
 6   Tesla_Occupant  294 non-null    int64
 7   Other_Vehicle   294 non-null    int64
 8   Cyclists_Peds   294 non-null    int64
 9   TSLA_cycl_peds  294 non-null    int64
 10  Model           294 non-null    object
 11  Claimed         294 non-null    int64
 12  VTAD            294 non-null    int64
dtypes: float64(1), int64(7), object(5)
memory usage: 32.2+ KB
```

```
df.head()
#So we have Date so derive the per year, per day
```

	Date	Country	State	Description	Deaths	Tesla_Driver	Tesla_Occupant	Other_Vehicle	Cyclists_Peds	TSLA_cycl_peds	Mo
0	1/17/2023	USA	CA	Tesla crashes into back of semi	1.0	1	0	0	0	1	
1	1/7/2023	Canada	-	Tesla crashes	1.0	1	0	0	0	1	
2	1/7/2023	USA	WA	Tesla hits pole, catches	1.0	0	1	0	0	1	

```
#Convert the Data column from Object to Date Column
df['Date'] = pd.to_datetime(df['Date'])
```

```
df.info()
#date is converted into datetime64(ns)
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 294 entries, 0 to 293
Data columns (total 13 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Date            294 non-null    datetime64[ns]
 1   Country         294 non-null    object
 2   State           294 non-null    object
 3   Description     294 non-null    object
 4   Deaths          294 non-null    float64
 5   Tesla_Driver    294 non-null    int64
 6   Tesla_Occupant  294 non-null    int64
 7   Other_Vehicle   294 non-null    int64
 8   Cyclists_Peds   294 non-null    int64
 9   TSLA_cycl_peds  294 non-null    int64
 10  Model           294 non-null    object
 11  Claimed         294 non-null    int64
 12  VTAD            294 non-null    int64
dtypes: datetime64[ns](1), float64(1), int64(7), object(4)
memory usage: 40.3+ KB
```

Exploratory Data Analysis

a. Perform an in-depth exploratory data analysis on the number of events by date, per year, and per day for each state and country

b. Analyze the different aspects of the death events. For example:

- What is the number of victims (deaths) in each accident?
- How many times did tesla drivers die?
- What is the proportion of events in which one or more occupants died?
- What is the distribution of events in which the vehicle hit a cyclist or a pedestrian?
- How many times did the accident involve the death of an occupant or driver of a Tesla along with a cyclist or pedestrian?
- What is the frequency of Tesla colliding with other vehicles?

a. Perform an in-depth exploratory data analysis on the number of events by date, per year, and per day for each state and country

```
#add the 3 columns event_year, event_month, event_day
df.loc[:,"event_year"] = df.Date.dt.year
df.loc[:,"event_month"] = df.Date.dt.month
df.loc[:,"event_day"] = df.Date.dt.day
```

```
df.columns
```

```
Index(['Date', 'Country', 'State', 'Description', 'Deaths', 'Tesla_Driver',
       'Tesla_Occupant', 'Other_Vehicle', 'Cyclists_Peds', 'TSLA_cycl_peds',
       'Model', 'Claimed', 'VTAD', 'event_year', 'event_month', 'event_day'],
      dtype='object')
```

```
#Year wise
```

Year wise

```
df.event_year.value_counts()
```

event_year	count
2022	93
2021	58
2019	46
2020	39
2018	18
2016	15
2017	11
2015	5
2014	4
2013	2

dtype: int64

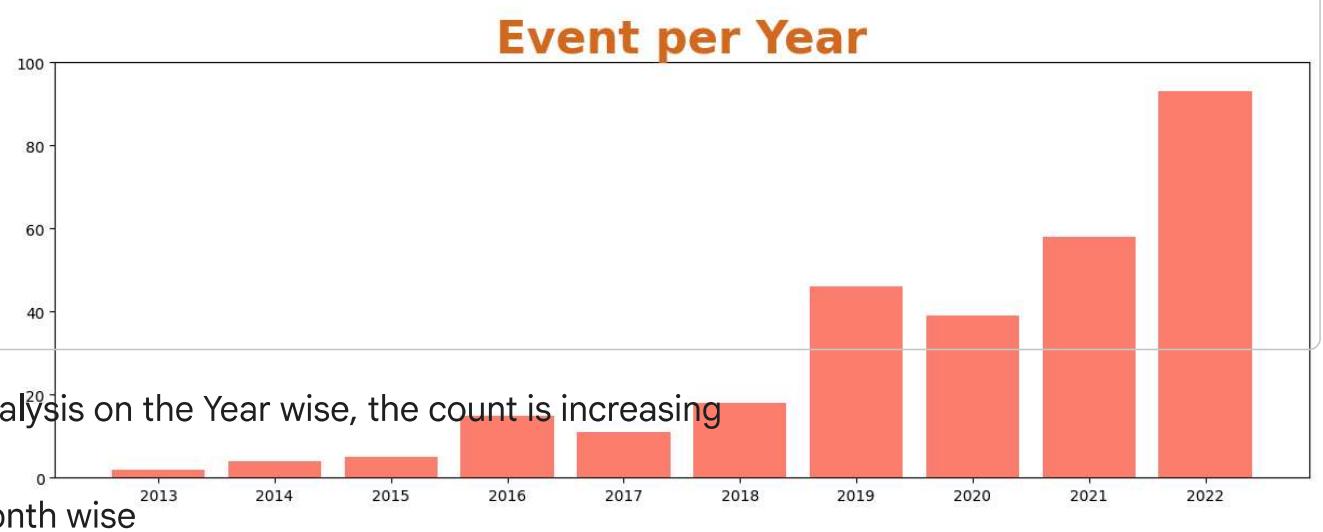
```
#Exclude the year 2023 since the value is less
df = df[df.event_year!= 2023]
```

```
vc = df.event_year.value_counts()
vc = vc.sort_index()
vc
```

event_year	count
2013	2
2014	4
2015	5
2016	15
2017	11
2018	18
2019	46
2020	39
2021	58
2022	93

dtype: int64

```
plt.figure(figsize = (15,5))
plt.bar(height = vc.values, x = vc.index, color = "salmon")
plt.xticks(vc.index, vc.index)
#for i in vc.index:
  #plt.annotate(vc[i], xy = (i-0.05, vc[i]+2), size = 13)
plt.ylim(0,100)
plt.title("Event per Year", size = 30, color = "chocolate", weight = "heavy")
plt.show()
```

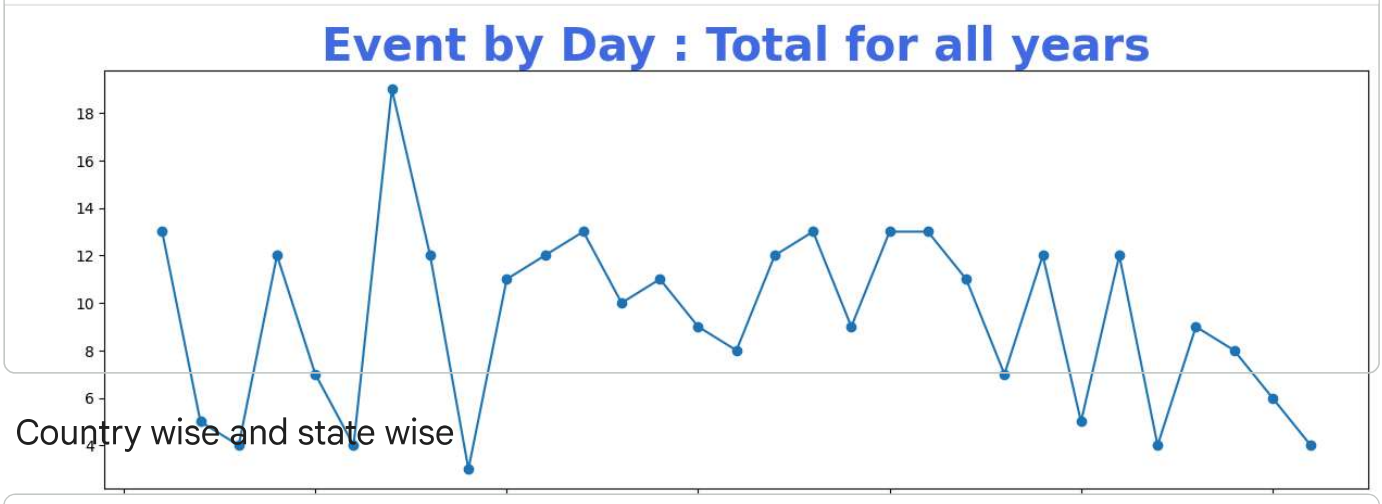


Analysis on the Year wise, the count is increasing

Month wise

```
mc = df.event_month.value_counts()
mc = mc.sort_index()

plt.figure(figsize = (15,5))
plt.bar(height = mc.values, x = mc.index, color = "red")
plt.xticks(mc.index, mc.index)
plt.ylim(0,100)
plt.title("Event per month", size = 30, color = "chocolate", weight = "heavy")
plt.show()
```


```
dc = df.event_day.value_counts()
dc = dc.sort_index()
plt.figure(figsize = (15,5))
plt.plot( dc.index, dc.values)
plt.scatter( dc.index, dc.values)
plt.xlabel("Day")
plt.title("Event by Day : Total for all years", size = 30, color = "royalblue",weight = "heavy")
plt.show()
```



Country wise and state wise

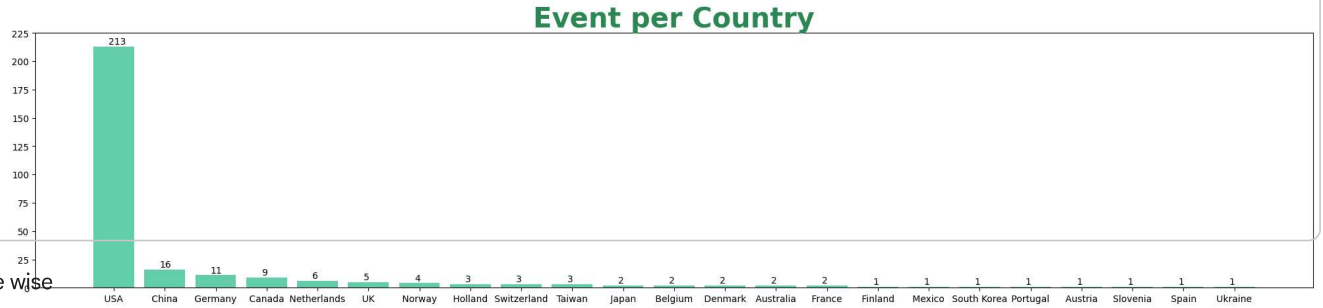
```
df["Country"].value_counts()
```

Country	count
USA	213
China	16
Germany	11
Canada	9
Netherlands	6
UK	5
Norway	4
Holland	3
Switzerland	3
Taiwan	3
Japan	2
Belgium	2
Denmark	2
Australia	2
France	2
Finland	1
Mexico	1
South Korea	1
Portugal	1
Austria	1
Slovenia	1
Spain	1
Ukraine	1

dtype: int64

```
yc = df.Country.value_counts()
yc = mc.sort_index()

yc = df.Country.value_counts()
plt.figure(figsize = (25,5))
plt.bar(height = yc.values, x = yc.index, color = "mediumaquamarine")
plt.xticks(yc.index, yc.index)
for i in range(len(yc.index)):
  plt.annotate(yc[i], xy = (i-0.1, yc[i]+2), size = 10)
plt.title("Event per Country", size = 30, color = "seagreen", weight = "heavy")
plt.ylim(0, 25 * round(yc.max()/25))
plt.show()
```

```
/tmp/ipykernel_3942/2173775872.py:9: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future ver
  plt.annotate(yc[i], xy = (i-0.1, yc[i]+2), size = 10)
```



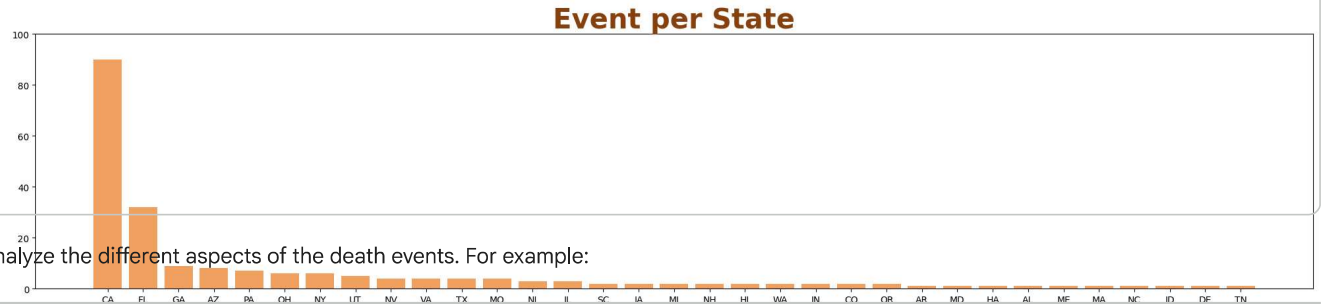
state wise

```
df.State = df.State.str.strip()
df["State"].value_counts()
```

State	count
CA	90
-	78
FL	32
GA	9
AZ	8
PA	7
OH	6
NY	6
UT	5
NV	4
VA	4
TX	4
MO	4
NJ	3
IL	3
SC	2
IA	2
MI	2
NH	2
HI	2
WA	2
IN	2
CO	2
OR	2
AR	1
MD	1
HA	1
AL	1
ME	1
MA	1

```
#remove - states
st = df.State.value_counts()
st = st[st.index != "-"]
```

```
plt.figure(figsize = (25,5))
plt.bar(height = st.values, x = st.index, color = "sandybrown")
plt.xticks(st.index, st.index)
plt.title("Event per State", size = 30, color = "saddlebrown", weight = "heavy")
plt.ylim(0, 25 * round(st.max()/25))
plt.show()
```



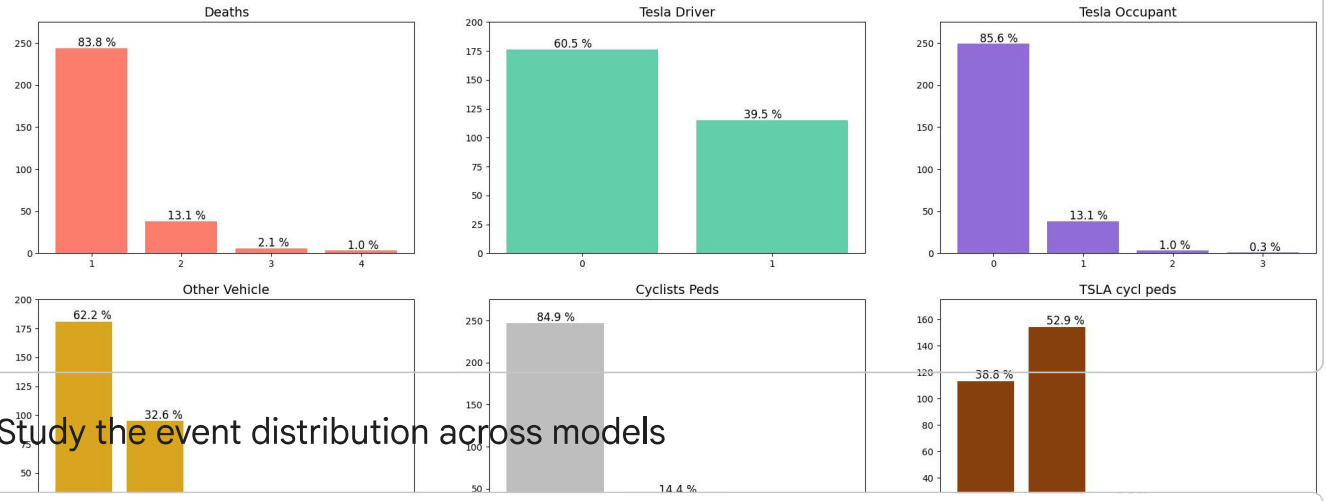
b. Analyze the different aspects of the death events. For example:

1. What is the number of victims (deaths) in each accident?
2. How many times did tesla drivers die?
3. What is the proportion of events in which one or more occupants died?
4. What is the distribution of events in which the vehicle hit a cyclist or a pedestrian?
5. How many times did the accident involve the death of an occupant or driver of a Tesla along with a cyclist or pedestrian?
6. What is the frequency of Tesla colliding with other vehicles

```
df.head()
```

	Date	Country	State	Description	Deaths	Tesla_Driver	Tesla_Occupant	Other_Vehicle	Cyclists_Peds	TSLA_cycl_peds	Model
3	2022-12-22	USA	GA	Tesla crashes and burns	1.0	1	0	0	0	1	-
4	2022-12-19	Canada	-	Tesla crashes into storefront	1.0	0	0	0	1	1	-
5	2022-	USA	GA	Tesla hit two	1.0	0	1	0	0	0	

```
col_list = ['Deaths', 'Tesla_Driver','Tesla_Occupant', 'Other_Vehicle', 'Cyclists_Peds', 'TSLA_cycl_peds']
color = ['salmon', 'mediumaquamarine', 'mediumpurple', 'goldenrod', 'silver', 'saddlebrown']
f,ax = plt.subplots(2,3, figsize = (25,10))
i,j,k = 0,0,0
for col in col_list:
  vc = df[col].value_counts()
  #print(vc)
  vc = vc.sort_index()
  perc = (vc/vc.sum()*100).round(1)
  #print(vc)
  ax[i,j].bar(x = vc.index, height = vc.values, color = color[k])
  ax[i,j].set_title(col.replace("_", " "), size = 14)
  ax[i,j].set_xticks(vc.index)
  for l in vc.index:
    ax[i,j].annotate("{} %".format(perc[l]), xy = (l-0.15,vc[l]+2), size = 12)
    ax[i,j].set_ylim(0,  25 * round(vc.max()/25)+25)
  j += 1
  k += 1
  if j == 3:
    j = 0
    i += 1
```



c. Study the event distribution across models

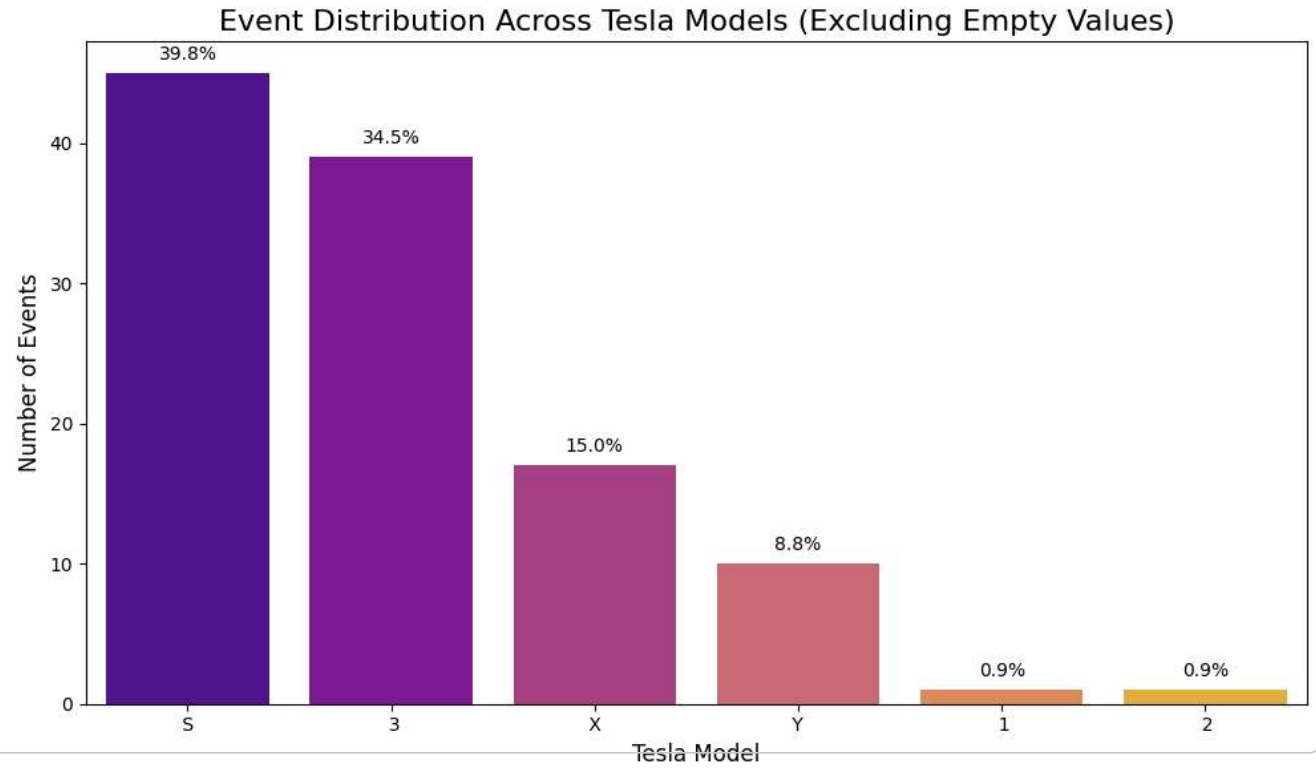
```
df.Model.value_counts()
#178 times no model are mentioned
```

Model	count
-	178
S	45
3	39
X	17
Y	10
1	1
2	1

dtype: int64

```
model_counts = df[df['Model'] != ' - ']['Model'].value_counts()

plt.figure(figsize=(10, 6))
ax = sns.barplot(x=model_counts.index, y=model_counts.values, palette='plasma')
plt.title('Event Distribution Across Tesla Models (Excluding Empty Values)', fontsize=16)
plt.xlabel('Tesla Model', fontsize=12)
plt.ylabel('Number of Events', fontsize=12)

for p in ax.patches:
    percentage = '{:.1f}%'.format(100 * p.get_height()/model_counts.sum())
    x = p.get_x() + p.get_width() / 2
    y = p.get_height()
    ax.annotate(percentage, (x, y), ha='center', va='bottom', fontsize=10, color='black', xytext=(0, 5), textcoords='offset poi

plt.tight_layout()
plt.show()

print("Event distribution across models (excluding empty values):")
print(model_counts)
```

```
/tmp/ipykernel_3942/145752667.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

  ax = sns.barplot(x=model_counts.index, y=model_counts.values, palette='plasma')
```




```
Event distribution across models (excluding empty values):
```

```
Model
3     45
3     39
```

d.Check the distribution of verified Tesla autopilot deaths

```
df.head()
```

```
1      1
2      1
Name: count, dtype: int64
```

	Date	Country	State	Description	Deaths	Tesla_Driver	Tesla_Occupant	Other_Vehicle	Cyclists_Peds	TSLA_cycl_peds	Model
3	2022-12-22	USA	GA	Tesla crashes and burns	1.0	1	0	0	0	1	-
4	2022-12-19	Canada	-	Tesla crashes into storefront	1.0	0	0	0	1	1	-
5	2022-	USA	GA	Tesla hit two	1.0	0	1	0	0	0	

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 291 entries, 3 to 293
Data columns (total 16 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Date            291 non-null    datetime64[ns]
 1   Country         291 non-null    object
 2   State           291 non-null    object
```